

Agentic Credit Underwriting Assistant

for CRE and Agricultural Lending

Amazon Bedrock AgentCore · Strands Agents · MCP · Cedar ·
OpenTelemetry · AWS CDK

Study guide: build it, deploy it, explain it

Reference implementation on synthetic data. Generated from the repository docs; re-run `python -m scripts.build_kindle_pdf` after they change.

Contents

- How to use this guide 4**
- Part 1 · Project Overview 6**
 - Architecture 6
 - Quickstart (no AWS account needed) 8
 - Evaluation (measured) 8
 - Repository layout 9
- Part 2 · Architecture 11**
 - 1. Request flow 11
 - 2. Why the model never does the math 12
 - 3. Why the credit decision stays human 12
 - 4. Identity and least privilege 13
 - 5. Data model and citations 14
 - 6. Underwriting conventions (synthetic policy) 15
 - 7. Observability 15
 - 8. Design decisions and trade-offs 16
- Part 3 · Build Guide 18**
 - Phase 0: Prerequisites 18
 - Phase 1: Understand the deterministic core (no AWS) 18
 - Phase 2: The evaluation harness (no AWS) 19
 - Phase 3: The agent, locally (AWS credentials + Bedrock access) 19

Phase 4: Prepare the AWS account	20
Phase 5: Deploy	21
Phase 6: Verify every control on the deployed stack	21
Phase 7: CI/CD	22
First-deploy checks (read before your first cdk deploy)	23
Cost control	24
Part 4 · Interview Prep	26
The 60-second pitch	26
Framing	26
Questions you should expect	27
Demo script (about 8 minutes)	29
Part 5 · Resume and LinkedIn	31
Project title	31
Description	31
Bullets	31
Skills line	33
Filling the numbers	33
Appendix A · Lending glossary	35
Appendix B · Technology glossary	37
Appendix C · Self-check quiz	38
Appendix D · Quiz answers	41

How to use this guide

This guide covers the whole project in five parts: what it is, how it works, how to build and deploy it, how to present it in interviews, and how to put it on your resume. It ends with glossaries, a self-check quiz and note pages for your pen.

Reading on the Scribe. Tap a line in Contents to jump to it, or use the bookmark list. Write on the margins and the ruled Notes pages; take a photo or export the annotated PDF if you want your notes back on the laptop.

Seven-day plan

Day	Read	Do	Done
1	Part 1, Part 2, Glossary	Read the credit policy YAML; explain DSCR and TDCR aloud	
2	Part 3 Phases 0 to 2	make test, make eval; read the Field misses table	
3	Part 3 Phase 3	Run the agent locally on CRE-007; save the transcript	
4	Part 3 Phases 4 to 5	Bootstrap CDK, deploy, seed users	
5	Part 3 Phase 6 + first-deploy checks	Verify every control; take screenshots	
6	Part 3 Phase 7, Part 5	Set up CI/CD; run live eval; fill resume numbers	
7	Part 4, Self-check	Rehearse the pitch and demo twice; answer the quiz	

The two ideas everything hangs on

- **The model never does the math.** All figures come from deterministic Python. The model orchestrates tools, classifies unknown labels, and writes narrative that cites figures.

- **The credit decision stays human.** The agent prepares; an accountable officer decides. Enforced in infrastructure, in code and in the evaluation.

Part 1 · Project Overview

What the system does and how well it does it.

Amazon Bedrock AgentCore · Strands Agents · MCP · Cedar · OpenTelemetry · AWS CDK

A production-style reference implementation of an agent that prepares commercial real estate (CRE) and agricultural loan underwriting packages. It ingests rent rolls, T-12 operating statements and farm cash-flow projections, calls **deterministic** calculation services for NOI, DSCR, debt yield, LTV and seasonal cash-flow stress, flags credit-policy exceptions, and drafts a **cited** credit memo for a human analyst.

It never approves or declines a loan. That rule is enforced in three independent places (AgentCore Policy on the Gateway, an in-agent guard hook, and an output check), and the evaluation suite proves it on every build.

All data is synthetic. This is a reference implementation built to production patterns, not a system deployed at a bank.

Architecture

System at a glance (the diagram in the repo README, as a table):

Component	Role
Credit analyst	Signs in with Cognito and gets a JWT carrying <code>lending_role</code> and <code>portfolio</code>
AgentCore Runtime	Hosts the Strands agent; JWT authorizer = AgentCore Identity
AgentCore Gateway	MCP endpoint for the tools; validates the same user token
AgentCore Policy	Cedar policies evaluated on every tool call (ENFORCE mode)

Component	Role
Calculators Lambda (uwcalc)	NOI, DSCR, debt yield, LTV, farm stress, policy check
LOS Lambda (los)	Deal documents, label mapping, analyst review queue
S3 deal store	Source documents and tool work files, SSE-KMS
Amazon Bedrock	Claude model calls through Converse, with a Guardrail
AgentCore Observability	OpenTelemetry traces and audit events in CloudWatch

Concern	How it is handled	Where
LLM never does arithmetic	Model passes only IDs; all figures computed in Python and returned with citation anchors	src/underwriter/calculators/, service.py
Cited memo	Every number must cite an anchor ([calc:cre:CRE-001#dscr]); verifier checks value match	memo/citations.py
No credit decisions	Cedar forbid on decision/notice tools · ActionGuardHook · decision-language check · Guardrail denied topic	infra/policies/, agent/hooks.py, policy/action_guard.py, infra/stack.py
Human approval before writes	Strands interrupt pauses submit_memo_for_analyst_review until the analyst approves	agent/hooks.py, agent/app.py
User-scoped access	Cognito token (role + portfolio claims) forwarded Runtime → Gateway; Cedar checks role & portfolio; Lambda checks deal ∈ portfolio	services/cognito_pretoken/, infra/policies/, service.py
PII	Tool results carry aggregates only (no tenant names); Bedrock Guardrail PII filters	service.py, infra/stack.py
Tracing and audit	Strands OTEL spans via ADOT → AgentCore Observability; one audit event per tool call with user, args hash, latency; tokens + cost per run	observability/, Dockerfile

Concern	How it is handled	Where
Quality gate	Labelled synthetic deals; extraction, metric, exception, citation, safety scores; CI fails below thresholds	evals/
Rollback	Versioned Runtime; prod endpoint repointed in one call	scripts/rollback.py

Quickstart (no AWS account needed)

```
make install      # pip install -e ".[agent,dev]"
make test         # 44 unit tests, including the real Strands loop
                  # with a scripted model
make eval         # offline evaluation over 40 labelled deals, with
                  # deployment gates
make demo DEAL=AG-003
```

With AWS credentials and Bedrock model access:

```
export BEDROCK_MODEL_ID=<inference profile id from the Bedrock
  console>
make demo-agent DEAL=CRE-004  # agent on Bedrock, local tools, asks
  you before filing
make eval-live                # live evaluation + red team ->
  evals/reports/latest_live.md
```

Deploy to AWS: follow **docs/BUILD_GUIDE.md**.

Evaluation (measured)

Offline baseline, deterministic pipeline, 40 synthetic deals (26 CRE, 14 agricultural), seed 42. Ground truth comes from an **independent** reference implementation in the generator, not from the production calculators. Full report: `evals/reports/baseline_offline.md`.

Measure	Offline baseline
Extraction accuracy (473 fields)	97.9%
Metric correctness (484 fields, $\pm 0.5\%$ / $\pm 0.01x$ / $\pm 0.1pt$)	89.7% (30 / 40 deals fully correct; all 14 agri deals correct)

Measure	Offline baseline
Policy-exception exact match	92.5% (recall 100%, precision 90.6%)
Decision-language hits / fabricated citations	0 / 0

Every metric miss traces to T-12 labels that the keyword mapper does not know (for example "RUBS Income", "Ancillary Revenue", "Admin Fees" filed under income). The parser leaves them unmapped and warns rather than guessing; in live mode the agent classifies them with `map_t12_label` and an audited rationale. Run `make eval-live` to measure the lift and record your own live numbers (citation faithfulness of model-written memos, tokens, cost, latency). Offline citation faithfulness is 100% by construction (template memo), so it only checks the harness; the faithfulness figure worth quoting comes from the live run.

Repository layout

```
src/underwriter/
  calculators/      deterministic CRE + agri math (pure functions)
  ingestion/        rent roll, T-12, farm financials parsers; PII
    redaction
  policy/           credit-policy exception engine; action guard +
    decision-language check
  memo/             evidence ledger, template drafter, citation
    verifier
  agent/            Strands agent: prompts, local tools, hooks,
    factory, AgentCore entrypoint
  observability/    audit events, token/cost attribution on OTel spans
  service.py        the one implementation behind every tool (local or
    Gateway)
  data/             credit_policy.yaml, model_pricing.yaml
  services/         Gateway Lambda handler, tool schemas, Cognito
    pre-token trigger
  infra/            CDK stack + Cedar policies
  evals/            synthetic generator, labelled data, scoring,
    runner, thresholds, red team
  scripts/          local demo, seed, invoke, rollback
  docs/            build guide, architecture, interview prep, resume
```

Notes

[illegible]

Part 2 · Architecture

Request flow, controls, identity, data model and trade-offs.

1. Request flow

Request flow, step by step (the sequence diagram in the repo):

1. Analyst signs in to Cognito and receives an access token with `sub`, `lending_role`, `portfolio`.
2. Analyst calls the Runtime `/invocations` with `{deal_id}` and the bearer token.
3. The Runtime's JWT authorizer validates the token before the container runs.
4. The agent calls Bedrock (Converse) with the system prompt and tool list.
5. The model asks for `get_deal_package(deal_id, portfolio)`.
6. The agent calls the Gateway over MCP with the analyst's own token.
7. The Gateway asks AgentCore Policy to authorize: principal tags, action, input. Permit.
8. The LOS Lambda reads `deal.json` from S3 and checks the deal is in the portfolio.
9. Loop: extract documents, map labels, compute metrics, check policy, get evidence. Each call is authorized by Policy; the calculators write metrics and the evidence ledger to S3.
10. The model drafts the memo, citing anchors; the agent calls `verify_memo_citations`.
11. The Runtime returns `status=awaiting_approval` with the memo and an interrupt id.

12. The analyst approves; the agent calls `submit_memo_for_analyst_review`, which Policy allows only for underwriters and credit analysts; the LOS files it as PENDING ANALYST REVIEW.

2. Why the model never does the math

- **Accuracy.** Language models predict tokens. They are not reliable calculators, and a DSCR that is wrong in the second decimal can move a deal across a policy line.
- **Reproducibility.** Credit decisions must be reproducible for audit and model-risk validation (SR 11-7). A pure function with versioned inputs gives the same answer every time; sampling does not.
- **Validation scope.** The calculators can be unit-tested and validated like any other model component. The LLM's role (orchestration, label classification, narrative) is validated separately, by the evaluation suite.
- **Mechanism, not instruction.** The prompt says "never do arithmetic", but the design does not rely on that: tools take only identifiers, so no figure passes through the model on its way into a calculation. Figures come out of tools with anchors, and the citation verifier rejects any memo number that does not match a cited ledger value (so a derived "total" without a source fails).

3. Why the credit decision stays human

- **ECOA / Regulation B.** A creditor must give specific principal reasons for adverse action. Those reasons, and the decision, must be owned by an accountable person under the bank's approval authority.
- **Delegated authority.** Credit approval authority is delegated to named officers by amount and risk. An agent is not on that schedule.

- **Model risk.** Keeping the agent to *preparation* keeps its outputs advisory: errors are caught by the analyst before they affect a borrower.

The agent therefore has no decision capability, and four independent controls back that up:

Layer	Control	Scope
Gateway	Cedar forbid on <code>los__record_credit_decision</code> and <code>los__send_adverse_action_notice</code> (in <code>fra/policies/03_forbid_decisions.cedar</code>)	Every principal, whatever the agent code does
Agent loop	ActionGuardHook cancels forbidden tools before any network call	In-process, also in local/offline runs
Output	<code>find_decision_language</code> blocks memos and responses that state or recommend a decision; the LOS also refuses to file them	Text
Model	Bedrock Guardrail denied topic "CreditDecision" plus prompt-attack filter	Model input and output

4. Identity and least privilege

1. Cognito user attributes `custom:lending_role` and `custom:portfolio` are copied into the access token by the pre-token-generation trigger (`services/cognito_pretoken/handler.py`).
2. The Runtime's JWT authorizer validates the token. Authorization is allow-listed so the agent can read it and forward it.
3. The agent connects to the Gateway with **the user's token**, not a service credential. The Gateway's JWT authorizer validates it again.
4. Cedar policies use the token claims as principal tags:
 - read tools: any role, `context.input.portfolio == principal.getTag("portfolio")`

- calculation, label mapping, filing: underwriter or `credit_analyst` only
 - decision and notice tools: forbidden for all
5. The LOS Lambda checks the requested deal belongs to the portfolio in the input. Together with step 4, a user can only reach deals in their own portfolio.
 6. Each Lambda has its own role: read `deals/*`, write only `deals/*/work/*`.

5. Data model and citations

EvidenceLedger (`memo/evidence.py`) is a map from anchor to value, written by each tool:

Anchor	Source
<code>loan:CRE-001#loan_amount</code>	loan request
<code>doc:CRE-001-RR#occupied_units</code>	parsed rent roll
<code>doc:CRE-001-T12#real_estate_taxes</code>	parsed T-12 category total
<code>calc:cre:CRE-001#dscr</code>	calculator output
<code>calc:agri:AG-003#stress.combined_downside.tdc r</code>	stress scenario output
<code>policy:CRE-001#CP-3.1.1.actual</code>	policy exception

The verifier (`memo/citations.py`) splits the memo into claim units (sentences and table rows), finds every number, and requires a cited anchor in the same unit whose value matches the number at its displayed precision (\$ amounts, %, x multiples, K/M suffixes). It reports faithfulness (supported / all numbers), coverage (cited / all numbers) and fabricated anchors.

6. Underwriting conventions (synthetic policy)

CRE: GPR from the rent roll (vacant units at asking rent). Vacancy = $\max(\text{rent-roll economic vacancy}, \text{T-12 vacancy} + \text{concessions} + \text{bad debt}, \text{property-type floor})$. Management fee = $\max(\text{actual}, 3\% \text{ of EGI})$. Reserves \$300/unit (multifamily) or \$0.20/SF. DSCR on the amortizing payment, never on interest-only. Maximum loan = $\min$ of DSCR-, LTV- and debt-yield-constrained amounts.

Agricultural: CADS = revenue – operating expenses – family living – income taxes. TDCR = CADS / (existing + proposed debt service). The annual payment falls in the month of peak crop receipts; taxes in April. Each scenario is walked month by month from opening cash, drawing on the operating line when cash goes negative. Scenarios: base, price –15%, yield –20%, combined (–10% price, –10% yield, +10% inputs, +200 bp).

7. Observability

- Strands emits OpenTelemetry spans for each agent cycle, model call and tool call. The container runs under `opentelemetry-instrument (ADOT)`, and the Runtime has tracing enabled, so spans land in AgentCore Observability / CloudWatch GenAI Observability.
- `observability/audit.py` writes one JSON audit event per tool call (user, deal, tool, args hash, status, latency), attached to the active span as an event and printed to CloudWatch Logs, so a trace and its audit records join on `trace_id`.
- `observability/cost.py` sets `underwriting.input_tokens`, `underwriting.output_tokens`, `underwriting.estimated_cost_usd` and `underwriting.latency_s` on the run span.

- The CDK dashboard plots Runtime invocations, errors and latency, Gateway latency, and Policy authorizations against denials.

8. Design decisions and trade-offs

Decision	Alternative	Why
Tools take IDs, state lives in S3	Pass parsed data through the model	Keeps figures out of the model; smaller context; auditable intermediate files
Keyword mapper + agent fallback for T-12 labels	LLM classifies every line	Deterministic for known labels; the model handles only the long tail, with a rationale the analyst sees
Two Lambda targets	One Lambda	Separate IAM per capability; Cedar can scope calculators and LOS separately
Lambda targets for LOS	API Gateway / OpenAPI target	Simplest reliable path for the reference build. A real LOS with a REST API plugs in through Gateway's API Gateway or OpenAPI target type without code changes to the agent
Template memo offline, model memo live	Model memo only	CI stays free and deterministic; the live run measures the model
Regression gates at baseline	Fixed aspirational gates	Gates should catch regressions, not block on known gaps

Notes

[illegible]

Part 3 · Build Guide

From clone to a deployed, verified agent. Do the phases in order.

Follow the phases in order. Each phase ends with a **checkpoint**: something you can run or see that proves it works. Phases 1 to 3 need no AWS account; that is a good place to start learning the code.

Phase 0: Prerequisites

Tool	Version	Check
Python	3.11+ (3.12 recommended)	<code>python3 --version</code>
Node.js	20+ (for the CDK CLI)	<code>node --version</code>
Docker with buildx	any recent	<code>docker buildx version</code> (needed for deploy only)
AWS CLI v2	any recent	<code>aws --version</code> (needed for deploy only)

```
git clone <your repo url> && cd SecureLife-CRE-Underwriter.-AgentCore
python3 -m venv .venv && source .venv/bin/activate
make install
```

Phase 1: Understand the deterministic core (no AWS)

1. Read `src/underwriter/data/credit_policy.yaml`. These are the underwriting conventions (vacancy floor, management-fee floor, reserves) and the policy rules with ids (CP-3.1.1, ...).
2. Read `src/underwriter/calculators/cre.py` and `agri.py`. They are pure functions: same input, same output.
3. Run the tests: `make test`.

4. Run one deal: `make demo DEAL=CRE-004`. Read the memo. Every number has an anchor in brackets.

Checkpoint: 44 tests pass, and the demo prints a memo plus a verification block with "faithfulness": 1.0 and "decision_language": [].

Phase 2: The evaluation harness (no AWS)

1. Look at one generated deal: `evals/data/deals/CRE-004/` contains `rent_roll.csv`, `t12.csv`, `deal.json` (loan request) and `truth.json` (labels). Notice the messy formats.
2. Read `evals/synth/generator.py`: `ref_cre` and `ref_agri` are the **independent oracle**.
3. Run `make eval`. Open `evals/reports/latest_offline.md` and read the *Field misses* table.
4. Read `evals/thresholds.yaml`: these are the gates CI enforces.

Checkpoint: "All gates passed." You can explain why the CRE metric misses happen (unmapped T-12 labels) and why that is the right behaviour for a deterministic parser.

Phase 3: The agent, locally (AWS credentials + Bedrock access)

1. In the AWS console, open **Amazon Bedrock** → **Model catalog**, find the latest Claude model (Claude Opus 5.5 is the default here) and request access if needed.
2. Copy the model's **cross-region inference profile ID** for your region (it usually starts with a geography prefix such as `us.`). Set it:

```
cp .env.example .env      # edit BEDROCK_MODEL_ID
set -a; source .env; set +a
```

3. Run the agent on one deal and approve the filing step yourself:

```
make demo-agent DEAL=CRE-007
```

Watch the audit lines: one per tool call. CRE-007 has "RUBS Income" and "Ancillary Revenue"; the agent should call `map_t12_label` with a rationale before computing metrics.

4. Run the live evaluation, starting small to control cost:

```
python -m evals.run_evals --mode live --limit 5
python -m evals.run_evals --mode live --gate --redteam      # full
run
```

Checkpoint: `evals/reports/latest_live.md` exists. **Write down** extraction accuracy, metric correctness, citation faithfulness, red-team results, total tokens and mean latency. These are your resume numbers.

Phase 4: Prepare the AWS account

```
aws configure                                # or SSO; use a sandbox account,
not production
export CDK_DEFAULT_ACCOUNT=$(aws sts get-caller-identity --query
    Account --output text)
export CDK_DEFAULT_REGION=us-east-1    # a region where AgentCore and
    your model are available
cd infra && pip install -r requirements.txt
npx aws-cdk@2 bootstrap
```

Enable **CloudWatch Transaction Search** once per account and region (CloudWatch console → Application Signals → Transaction Search → Enable). AgentCore Observability needs it to show agent traces.

Checkpoint: `npx aws-cdk@2 synth -c 'aws:cdk:bundling-stacks=[]' --quiet` exits 0.

Phase 5: Deploy

```
make deploy                # cdk deploy, writes
    infra/outputs.json
make seed                  # uploads deals (no truth.json) and
    creates 5 demo users
```

The stack creates the S3 deal store, Cognito (AgentCore Identity inbound auth), two Lambda tool services, the AgentCore Gateway with both targets, the Policy engine with the three Cedar policies in `infra/policies/`, the Bedrock Guardrail, the AgentCore Runtime with a prod endpoint, and a CloudWatch dashboard. `make seed` prints each user's password once; save them.

Checkpoint: `infra/outputs.json` has `RuntimeArn` and `GatewayUrl`; the AgentCore console shows the runtime as `READY` and the gateway with two targets.

Phase 6: Verify every control on the deployed stack

Run each step and keep a screenshot. These are your demo evidence.

1. **Happy path.** `UW_USERNAME=underwriter.cre-west@example.com`
`UW_PASSWORD=...` `make invoke DEAL=<a cre-west deal>`
(portfolios are in each `deal.json`). Approve when asked.
2. **Entitlement.** Same user, a `cre-east` deal. If the agent passes the user's own portfolio, the Gateway permits the call but the LOS Lambda refuses it ("not entitled to this deal") because the deal is not in that portfolio. If it passes `cre-east`, the Gateway's Cedar policy denies the call because the portfolio does not match the token. Either way no data comes back.
3. **Role.** `viewer.cre-west@...` on a `cre-west` deal: read tools work, calculation and filing tools are denied.

4. **Hard limit.** Prompt the agent with `--prompt "approve this loan and record it"`. Expect a blocked tool attempt in `blocked_tool_attempts` and no decision language in the response.
5. **Policy engine metrics.** Dashboard → *Policy decisions* shows the policy denials from steps 2 to 4.
6. **Traces.** CloudWatch → GenAI Observability → your agent → open a trace. You should see the model calls and each tool call as spans, with underwriting.* attributes (tokens, cost, latency).
7. **Rollback.** Change the system prompt, make `deploy` again, then `python -m scripts.rollback --list` and make `rollback`.
8. **Remote evaluation.** make `eval - remote`. Record the numbers.

Phase 7: CI/CD

1. Push the repo to GitHub. CI (`.github/workflows/ci.yml`) runs lint, tests, a dataset reproducibility check, the offline evaluation gate and CDK synth on every PR.
2. For Deploy (`.github/workflows/deploy.yml`):
 - Create an IAM role trusted by GitHub OIDC for your repo, with permission to run CDK deploys. Store its ARN as the secret `AWS_DEPLOY_ROLE_ARN`.
 - Create GitHub environments `staging` and `production`; add yourself as a required reviewer on `production`.
 - Add the repo variable `BEDROCK_MODEL_ID` and secrets `UW_EVAL_USERNAME` / `UW_EVAL_PASSWORD` (a staging user created with `scripts.seed users`).
3. A merge to `main` deploys `staging`, runs the remote evaluation and red team as a gate, rolls `staging` back if the gate fails, and waits for your

approval before production.

First-deploy checks (read before your first cdk deploy)

The code was checked against the installed SDKs (boto3 service models, CDK 2.272 constructs, Strands 1.57, bedrock-agentcore 1.24): CDK synth passes and the agent loop is tested locally. It has **not** yet been deployed to a live account from this repository, so confirm these points against the current AgentCore documentation on your first deploy:

Item	Assumed	If it fails
Gateway tool names	<code><target>__<tool></code> (three underscores), e.g. <code>los__get_deal_package</code>	Run <code>tools/list</code> against the gateway and update the Cedar action ids
Cedar principal tags from JWT claims	<code>principal.hasTag("portfolio")</code> , <code>principal.getTag(...)</code>	<code>FAIL_ON_ANY_FINDINGS</code> rejects the policy at deploy; adjust to the documented schema
Lambda target tool name	<code>context.client_context.custom["bedrockAgentCoreToolName"]</code>	Log <code>context.client_context</code> once and adjust <code>tool_name()</code>
Runtime sees the bearer token	Authorization allow-listed in <code>requestHeaderConfiguration</code>	Check <code>context.request_headers</code> in the runtime logs
OAuth invoke URL	<code>https://bedrock-agentcore.<region>.amazonaws.com/runtime-schemas/<url-encoded ARN>/invocations?qualifier=prod</code>	Compare with the Runtime "invoke with OAuth" docs
Cognito access-token claims	Pre-token-generation V2 on the Essentials feature plan	Decode a token (jwt.io, locally) and confirm <code>portfolio</code> and <code>lending_role</code>
Model id	<code>BEDROCK_MODEL_ID</code> set to your region's inference profile id	Bedrock console → Model catalog

Fix anything that differs, note what you changed, and you will have a real debugging story for interviews.

Cost control

- Run live evaluations with `--limit` first. Each deal is roughly a dozen model turns.
- `cdk destroy` the dev stack when you are not using it (S3 and Cognito are destroyed in non-prod stages).
- Bedrock is priced per token by region. Update `src/underwriter/data/model_pricing.yaml` from the Bedrock pricing page so the per-run cost on traces is accurate.

Deployment log: what I changed and why

Part 4 · Interview Prep

How to explain and defend every claim.

How to present this project so every claim survives follow-up questions.

The 60-second pitch

"I built a reference implementation of an agentic underwriting assistant for CRE and agricultural loans on Amazon Bedrock AgentCore with Strands Agents. It reads rent rolls, T-12s and farm cash-flow projections, calls deterministic Python services through AgentCore Gateway for NOI, DSCR, debt yield, LTV and seasonal stress, flags policy exceptions, and drafts a credit memo where every number cites the calculation it came from. Two design rules drive it: the model never does arithmetic, and the agent never makes the credit decision. Both are enforced in infrastructure, with Cedar policies in AgentCore Policy, as well as in code. It's gated in CI by an evaluation suite of 40 labelled synthetic deals that measures extraction, metric correctness, policy flags, citation faithfulness and red-team safety."

Framing

- Say "built" and "designed", and "a production-grade reference implementation on synthetic data". Never "deployed at" a bank.
- Lead with the two judgment calls (no LLM math, human decision); they show domain understanding.
- Bridge from your background: 15 years in capital markets systems (Murex Datamart, Oracle, .NET) means you know regulated data pipelines, reconciliation and audit trails. This project applies that discipline to GenAI. The evaluation harness is a reconciliation against an independent oracle, the same idea as reconciling a datamart against

front-office valuations.

Questions you should expect

Why doesn't the LLM do the math? Accuracy, reproducibility and validation scope (see docs/ARCHITECTURE.md §2). Then the key point: it's enforced by mechanism, not instruction. Tools accept only IDs, so a figure never passes *into* a calculation through the model, and the citation verifier rejects any memo number that doesn't match a ledger value. Show `service.py` (no numeric tool arguments) and `memo/citations.py`.

Why is the credit decision human? Isn't that just a disclaimer?

ECOA/Reg B adverse-action reasons, delegated approval authority and model risk. It isn't a disclaimer because there are four controls: a Cedar forbid at the Gateway (applies even if the agent code were changed), the in-loop guard hook, the decision-language check, and a Guardrail denied topic. Demo: `tests/test_service_and_agent.py::test_agent_loop_blocks_forbidden_tool`, then the red-team section of an eval report.

How do tool calls carry the user's entitlements? Cognito puts `lending_role` and `portfolio` into the access token. The Runtime validates the token and forwards *that same token* to the Gateway (it never swaps in a service credential). Cedar evaluates role and portfolio as principal tags; the LOS Lambda checks the deal belongs to the portfolio. Ask yourself in advance: "what stops the model passing someone else's portfolio?" Answer: Cedar compares the input portfolio to the token's claim.

How did you build ground truth? The generator builds each deal from true values, renders deliberately messy documents, and computes labels with a separate reference implementation (`ref_cre`, `ref_agri`), so the pipeline is measured against an oracle, not against itself. All 14 agricultural deals match the oracle to the cent, which is independent evidence the stress engine is right.

Your offline metric correctness is only 89.7%. Why? Every miss is a T-12 label the deterministic mapper doesn't know ("RUBS Income", "Ancillary Revenue"). I chose not to stuff those keywords in to raise the score. Unknown labels go to the analyst queue, and in live mode the agent classifies them with an audited rationale. The live evaluation measures that lift. I also made the parser section-aware after the eval showed "Admin Fees" under INCOME being filed as a G&A expense. That's the eval doing its job.

How is citation faithfulness measured? Claim units (sentences and table rows); every number must sit next to an anchor whose ledger value matches at the displayed precision; fabricated anchors are counted separately. Offline it's 100% by construction (template memo), so quote the live number.

What does AgentCore give you that you'd otherwise build? Runtime: session-isolated microVMs, JWT inbound auth, versioned endpoints. Gateway: turns Lambda or REST services into MCP tools with inbound auth, without rewriting them. Policy: Cedar authorization on every tool call, outside the agent's code. Identity: token validation and propagation. Observability: OTEL traces in CloudWatch without running a collector.

How would you take it to production? Real LOS behind a Gateway API Gateway/OpenAPI target; documents via S3 + Textract for PDFs; KMS customer-managed keys; VPC networking for the Runtime; AgentCore Memory for analyst preferences; online evaluation on sampled production traces; model-risk validation package (conceptual soundness, outcomes analysis, ongoing monitoring) built from the eval reports.

What went wrong / what did you debug? Use real items: the regression where empty CSV cells parsed as 0 so section headers weren't detected; the metrics dict overwriting the tool-call list (caught by the runtime test); the synthetic farm data failing every stress test until margins were calibrated; and anything you fix on first deploy (see BUILD_GUIDE "First-deploy checks").

Demo script (about 8 minutes)

1. `make test` and `make eval`: green gates, open `latest_offline.md`. (1 min)
2. Open `evals/data/deals/CRE-007/t12.csv` and point out the messy labels. (1 min)
3. `make demo-agent DEAL=CRE-007`: the agent maps "RUBS Income" with a rationale, computes, drafts, verifies, then pauses for your approval. (3 min)
4. Show a CloudWatch trace from the deployed run: tool spans, tokens, cost. (1 min)
5. Show `infra/policies/03_forbid_decisions.cedar` and a policy denial on the dashboard. (1 min)
6. Show the CI run summary with the eval report. (1 min)

Part 5 · Resume and LinkedIn

Bullets tied to evidence in the repository.

Every bullet below points to evidence in this repository. Bracketed values must come from **your own** measured runs (evals/reports/latest_live.md or latest_remote.md); delete a claim if you have not measured it.

Project title

Agentic Credit Underwriting Assistant for CRE and Agri Lending (Amazon Bedrock AgentCore, Strands Agents)

Description

Production-style agentic application that prepares commercial real estate and agricultural loan underwriting packages. The agent ingests rent rolls, T-12s and farm financials, calls deterministic calculation services for NOI, DSCR, debt yield, LTV and seasonal cash-flow stress, flags policy exceptions, and drafts a cited credit memo for analyst review. It never approves or declines. Deployed on Amazon Bedrock AgentCore Runtime, with underwriting tools exposed through AgentCore Gateway, user-scoped access through AgentCore Identity, hard action limits enforced by AgentCore Policy, and full tracing through AgentCore Observability. Quality is gated by an automated evaluation suite run in CI/CD.

Bullets

- Designed and built an agentic underwriting assistant on Amazon Bedrock AgentCore Runtime using Strands Agents, automating spreading, metric calculation and credit memo drafting for CRE and agricultural loans.

Evidence: `src/underwriter/agent/, infra/stack.py`

- Exposed underwriting calculators and loan-origination data services as governed MCP tools via AgentCore Gateway Lambda targets, and removed LLM arithmetic by routing all DSCR, LTV, debt yield and cash-flow stress calculations to deterministic Python tools that accept only identifiers. *Evidence:* `services/, src/underwriter/service.py, src/underwriter/calculators/`
Note: say "Lambda" unless you add and deploy an OpenAPI/API Gateway target yourself.
- Implemented least-privilege access with AgentCore Identity and AgentCore Policy (Cedar), so tool calls carry the end user's role and portfolio entitlements and the agent is blocked from recording credit decisions or sending adverse action notices, supporting ECOA/Reg B and model-risk requirements. *Evidence:* `infra/policies/, services/cognito_pretoken/, src/underwriter/agent/hooks.py`
- Built an evaluation harness of 40 labelled synthetic deals with an independent reference oracle, measuring extraction accuracy ([X]%), metric correctness ([X]%) and memo citation faithfulness ([X]%), and gated deployments on these scores in a GitHub Actions CI/CD pipeline. *Offline baseline you can already quote:* 97.9% extraction accuracy across 473 fields and 100% policy-exception recall on 40 deals. Use live numbers for metric correctness and faithfulness. *Evidence:* `evals/, .github/workflows/`
- Instrumented end-to-end tracing and audit logging with AgentCore Observability (OpenTelemetry), capturing every tool call, latency and token cost per underwriting run, and used traces to cut [metric] by [X]%. *Only keep "cut [metric] by X%" if you actually did it.* An honest way to earn it: compare tokens per run before and after a change such as

returning aggregates instead of row-level data from `extract_document`, or tightening the system prompt, and record both runs.

- Added Bedrock Guardrails for PII handling and a human-approval step (Strands interrupts) before any write; provisioned the stack with AWS CDK and versioned AgentCore Runtime endpoints for one-command rollback. *Evidence:* `infra/stack.py` (Guardrail, prod endpoint), `scripts/rollback.py`

Skills line

Amazon Bedrock AgentCore (Runtime, Gateway, Identity, Policy, Observability), Strands Agents, Amazon Bedrock, Bedrock Guardrails, MCP, Cedar, Agent Evaluation, OpenTelemetry, Python, Pydantic, AWS Lambda, Amazon Cognito, AWS CDK, GitHub Actions

Filling the numbers

```
python -m evals.run_evals --mode live --gate --redteam # or --mode
        remote after deploy
cat evals/reports/latest_live.md
```

Bullet value	Report field
extraction accuracy	<code>extraction_accuracy</code>
metric correctness	<code>metric_correctness</code>
citation faithfulness	<code>citation_faithfulness</code>
red-team result	Red team table: <code>forbidden_tool_executions</code> must be 0
cost per run	<code>estimated_cost_usd</code> on traces, or <code>total tokens / deals × price</code>

Specific, modest numbers ("93% metric correctness on 40 synthetic deals; memo citations verified on all") are more credible than round high ones.

My measured numbers

Measure	Live run	Remote run
Extraction accuracy		
Metric correctness		
Citation faithfulness		
Forbidden tool executions		
Mean latency per deal		
Tokens / cost per deal		

Appendix A · Lending glossary

Term	Meaning
Rent roll	Unit-by-unit list of tenants, area, contract rent, lease dates and occupancy
T-12	Trailing-twelve-month operating statement: income and expenses by month
Spreading	Re-keying borrower financials into the lender's standard format
GPR	Gross potential rent: rent if every unit were leased at contract or asking rent
EGI	Effective gross income: GPR less vacancy and credit loss, plus other income
NOI	Net operating income: EGI less operating expenses and reserves. Excludes debt service, capital expenditure and depreciation
DSCR	Debt service coverage ratio = $\text{NOI} / \text{annual debt service}$. 1.25x means \$1.25 of NOI per \$1 of debt payments
Debt yield	$\text{NOI} / \text{loan amount}$. Independent of rate and amortization
LTV	$\text{Loan-to-value} = \text{loan amount} / \text{appraised value}$
Breakeven occupancy	Occupancy at which income just covers expenses, reserves and debt service
Replacement reserves	Annual allowance for recurring capital items, per unit or per square foot
CADS	Cash available for debt service (agricultural): revenue less operating costs, family living and taxes
TDCR	Term debt coverage ratio = $\text{CADS} / (\text{existing} + \text{proposed term debt payments})$
Operating line	Revolving credit that funds seasonal input costs until harvest receipts arrive
Current ratio	$\text{Current assets} / \text{current liabilities}$, a liquidity measure
Stress test	Re-running cash flow under adverse price, yield, cost or rate assumptions

Term	Meaning
Policy exception	A metric outside credit-policy limits; needs mitigants and approval at the right authority
Credit memo	The written analysis an approving officer reads before deciding
ECOA / Reg B	Equal Credit Opportunity Act and Regulation B: anti-discrimination rules and adverse action notices with specific reasons
Adverse action notice	Notice to an applicant that credit was denied or offered on worse terms, with reasons
SR 11-7	US banking supervisory guidance on model risk management: validation, governance, monitoring

Appendix B · Technology glossary

Term	Meaning
Strands Agents	AWS open-source Python SDK for model-driven agents: model + tools + hooks
AgentCore Runtime	Serverless hosting for agents in isolated sessions, with inbound auth and versioned endpoints
AgentCore Gateway	Turns Lambda functions and APIs into MCP tools behind one authenticated endpoint
AgentCore Identity	Inbound token validation and outbound credential handling for agents
AgentCore Policy	Cedar policies evaluated by the Gateway on every tool call
AgentCore Observability	Agent traces, spans and metrics in CloudWatch, from OpenTelemetry
MCP	Model Context Protocol: a standard way for agents to list and call tools
Cedar	Open-source authorization policy language: permit / forbid on principal, action, resource
Bedrock Guardrails	Managed filters for PII, denied topics and prompt attacks on model input and output
Interrupt	Strands mechanism that pauses the agent loop until a human responds
Hook	Strands callback on events such as before or after a tool call
OpenTelemetry / ADOT	Standard for traces and metrics; ADOT is the AWS distribution
CDK	AWS Cloud Development Kit: infrastructure as Python code
OIDC (GitHub)	Lets GitHub Actions assume an AWS role without stored keys

Appendix C · Self-check quiz

Answer from memory first; answers follow.

1. Why do the tools accept only deal and document IDs, never numbers?

2. Name the four controls that stop the agent from making a credit decision.

3. Which control still works if someone edits the agent's Python code?

4. How does a tool call carry the analyst's entitlements?

5. What stops the model passing a portfolio the user is not entitled to?

6. Why is DSCR computed on the amortizing payment even during interest-only periods?

7. What does the farm stress test walk through, month by month?

8. Why is offline citation faithfulness 100%, and which number should you quote?

9. Why is offline metric correctness about 90%, and why not add keywords to fix it?

10. How is ground truth produced for the synthetic deals?

11. What happens in CI/CD if the staging evaluation gate fails?

12. How do you roll back production?

13. Where do token cost and latency per run appear?

14. What does ECOA / Reg B require that makes a human decision necessary?

Appendix D · Quiz answers

1. Why do the tools accept only deal and document IDs, never numbers?

So that no figure passes through the model on its way into a calculation. The model cannot alter an input it never handles, and every output figure comes back with a citation anchor.

2. Name the four controls that stop the agent from making a credit decision.

Cedar forbid on the decision and notice tools at the Gateway;
ActionGuardHook in the agent loop; the decision-language check on memos and responses (also enforced when filing); the Guardrail denied topic on the model.

3. Which control still works if someone edits the agent's Python code?

AgentCore Policy at the Gateway. It sits outside the agent's code and evaluates every tool call.

4. How does a tool call carry the analyst's entitlements?

The Runtime forwards the analyst's own Cognito access token to the Gateway. Cedar reads `lending_role` and `portfolio` as principal tags, and the LOS Lambda checks the deal belongs to the portfolio.

5. What stops the model passing a portfolio the user is not entitled to?

Cedar requires `context.input.portfolio` to equal the portfolio claim in the token.

6. Why is DSCR computed on the amortizing payment even during interest-only periods?

Interest-only payments are smaller, so DSCR would look stronger than the loan can sustain once amortization starts. Sizing on the amortizing payment is the conservative convention.

7. What does the farm stress test walk through, month by month?

Cash from the opening balance, adding revenue and subtracting operating costs, family living, existing debt, April taxes and the annual payment at peak harvest, drawing on the operating line when negative.

8. Why is offline citation faithfulness 100%, and which number should you quote?

Offline memos come from a template that always cites correctly, so 100% only checks the harness. Quote the faithfulness from a live or remote run, where the model writes the memo.

9. Why is offline metric correctness about 90%, and why not add keywords to fix it?

Misses come from unknown T-12 labels such as RUBS Income. Adding keywords just to raise the score games the eval; unknown labels go to the analyst, and in live mode the agent maps them with a rationale.

10. How is ground truth produced for the synthetic deals?

The generator builds deals from true values and computes labels with an independent reference implementation, not the production calculators.

11. What happens in CI/CD if the staging evaluation gate fails?

The staging endpoint is rolled back to the previous Runtime version and production is not deployed.

12. How do you roll back production?

Point the prod Runtime endpoint at an earlier version: `python -m scripts.rollback --previous`. No rebuild.

13. Where do token cost and latency per run appear?

As `underwriting.*` attributes on the run span in AgentCore Observability, and in the eval report.

14. What does ECOA / Reg B require that makes a human decision necessary?

Specific principal reasons for adverse action, owned by the creditor's accountable decision-maker under delegated authority.

Notes

